An infinite family with one-sided l1-approximate cut dimension 2n minus 2

Troy Lee, Tongyang Li, Miklos Santha, Shengyu Zhang

doi:10.4230/LIPIcs.CCC.2021.15; CC BY-NC-SA 4.0

Editorial note. Complete native CC BY-NC-SA4.0 author version used for authorized noncommercial research delivery, retaining actual NC/SA terms. The primary selected construction is the original unnumbered Section7 result for n=3k+1 underlying authorTheorem3; no new extension to other n supplied. Original theorem/section offsets restored and all source-local proof dependencies retained. The scalar-query consequence is context within the same record, not an extra task. Native inline diagrams preserved with available tkz-graph; author matrices and perturbation budgets unchanged. All production/validation solely on Yupeng.

Allowed standard prerequisites (editorial).

- Weighted undirected graphs, cuts, linear span, laminar families and rooted tree representations. Complete original graph/edge-slot/weight/crossing definitions and source-local uncrossing/tree/linear-independence proofs retained; Jain lemma included with full author appendix proof

1 Introduction

Let $G = (V, w)$ be a weighted undirected n-vertex graph where w is an $\binom{n}{2}$-dimensional nonnegative real vector assigning a (possibly zero) weight to each edge slot. For a nontrivial subset $\emptyset \neq X \subsetneq V$, let $\Delta(X)$ be the set of edges of G with one endpoint in X and one endpoint in $\bar{X} = V \setminus X$. A *cut* S in G is a subset of edges of the form $\Delta(X)$ for a nontrivial set X. The sets X and $\bar{X}$ are called the *shores* of the cut. For a cut S, its *weight* is the sum of the weights of the edges in S, denoted $w(S)$. The minimum cut problem is to find the minimum of $w(S)$ over all cuts S. The study of algorithms for the minimum cut problem in theoretical computer science goes back at least to the 1960's and has given rise to a vast and beautiful literature. Minimum cut is also a problem of great practical importance with applications to, for example, clustering algorithms and evaluating network reliability. Randomized algorithms can solve the minimum cut problem in nearly linear time: in 1996 Karger gave an algorithm with running time $O(m \log^3(n))$ to compute the minimum cut of a weighted graph with m edges [Kar00]. This was the best known bound until very recently when two independent works

improved on it. Gawrychowski, Mozes, and Weimann [GMW20a] gave a randomized algorithm with running time $O(m \log^2(n))$ [GMW20a] and Mukhopadhyay and Nanongkai [MN20] gave a randomized algorithm with time complexity $O(m\frac{\log^2(n)}{\log\log n} + n\log^6(n))$. Gawrychowski, Mozes, and Weimann [GMW20b] later improved the running time of the Mukhopadhyay and Nanongkai algorithm to $O(m\frac{\log^2(n)}{\log\log n} + n\log^{3+\varepsilon}(n))$.

For *simple* graphs G, randomized algorithms are known with running times $O(m\log(n))$ and $O(m + n\log^3(n))$ [GNT20]. For simple graphs even nearly linear time *deterministic* algorithms are known. Kawarabayashi and Thorup gave an $O(m\log^{12}(n))$ time algorithm [KT19], which was subsequently improved to $O(m(\log(n)\log\log n)^2)$ by Henzinger, Rao, and Wang [HRW20].

Our work spans two aspects of the study of the minimum cut problem. The first is to query complexity lower bounds on minimum cut. A natural model in which to study the query complexity of minimum cut is for algorithms allowed to make *cut queries*. A cut query algorithm can query any subset $\emptyset \neq X \subsetneq V$ and receives the answer $w(\Delta(X))$. One motivation to study cut query algorithms comes from submodular function minimization. The cut function $f(X) = w(\Delta(X))$ is a submodular function, and finding the minimum cut value is equivalent to finding the minimum value of f over all nontrivial sets X. The problem of minimizing a submodular function is often studied with respect to an evaluation oracle, which in the case of the cut function is exactly a cut query.

Harvey [Har08] observed that results on the deterministic communication complexity of deciding graph connectivity [HMT88] imply that any deterministic cut query algorithm to compute minimum cut, or even to decide if the graph is connected or not, must make at least cn cut queries, for a constant $c < 1$. Analogous results on the randomized communication complexity of connectivity [BFS86] imply an $\Omega(n/\log(n))$ lower bound on the number of cut queries needed by a randomized algorithm to compute minimum cut (or even connectivity).

On the algorithms side, Rubinstein, Shramm, and Weinberg [RSW18] gave a randomized algorithm computing the minimum cut of a simple graph with $O(n\log(n)^3)$ many cut queries[1]. Recently, Mukhopadhyay and Nanongkai [MN20] used a different approach based on Karger's 2-respecting tree algorithm [Kar00] to also give a randomized $\tilde{O}(n)$ cut query algorithm to compute minimum cut in a general undirected weighted graph.

For deterministic cut query algorithms, there remains a large gap between the best upper and lower bounds. We are not aware of any deterministic algorithm for minimum cut better than learning the entire graph, which can take $\Omega(n^2/\log(n))$ cut queries in the worst case. On the lower bound side, Graur, Pollner, Ramaswamy, and Weinberg [GPRW20] recently introduced a very interesting lower bound technique called the *cut dimension*, which we now describe. Let $G = (V, w)$ be a weighted undirected graph with n vertices and m edges, and let $\mathcal{M}(G)$ be the set of minimum cuts of G. For a cut $S \in \mathcal{M}(G)$, let $\chi(S) \in \{0,1\}^m$ be the characteristic vector of S amongst the m edges of G. Let $\vec{\mathcal{M}}(G) = \{\chi(S) : S \in \mathcal{M}(G)\}$. The cut dimension of G, denoted $\mathrm{cdim}(G)$, is the dimension of $\mathrm{span}(\vec{\mathcal{M}}(G))$. It is shown in [GPRW20] that for any n-vertex graph G, the cut dimension $\mathrm{cdim}(G)$ is a lower bound on the deterministic cut query complexity of computing minimum cut on weighted n-vertex graphs. Moreover,

[1] [RSW18] state the bound as $\tilde{O}(n)$ but we estimate their sparsifier based algorithm to make $O(n\log(n)^3)$ many cut queries.

for every $n \geq 2$ they construct an n-vertex graph G with cut dimension $3n/2-2$.

Besides showing lower bounds on cut query complexity, the cut dimension is a natural measure of the complexity of mincuts in a graph. There is a rich literature on the possible structure of mincuts in a graph. Perhaps the first result of this kind is the cactus representation of mincuts by [DKL76]. A cactus for a graph G is a sparse weighted graph C that represents all the mincuts of G. One consequence of the cactus representation is that the number of possible mincuts in an n-vertex weighted graph is at most $\binom{n}{2}$. This upper bound was later given an algorithmic proof via Karger's famous contraction algorithm ([Kar93], Theorem 6.1). The n-vertex cycle graph has $\binom{n}{2}$ many minimum cuts and shows that this bound can be tight.

While the cycle has $\binom{n}{2}$ many mincuts, these cuts live in an n-dimensional space as the n-vertex cycle only has n edges. Is it possible to construct graphs with many cuts that also have high cut dimension? We show that this is not possible, and in fact the cut dimension of an n-vertex graph is at most $2n-3$.

Theorem 1 (Main Upper Bound)**.** *For any weighted undirected graph G on $n \geq 2$ vertices it holds that* $\mathrm{cdim}(G) \leq 2n-3$.

Like the cactus representation, this shows another aspect in which the mincuts of a graph are constrained to have a relatively simple structure. We further show that this bound is tight by constructing graphs with cut dimension $2n-3$ for every $n \geq 2$.

Theorem 2 (Main Lower Bound)**.** *For every $n \geq 2$ there exists an n-vertex weighted undirected graph G with* $\mathrm{cdim}(G) = 2n-3$.

In addition to shedding further light on the structure of minimum cuts, this improves the best known lower bound on the deterministic cut query complexity of the minimum cut problem to $2n-3$. We additionally show that the cut dimension is even a lower bound on a stronger query model called the linear query model, recently studied in [ACK20]. In the linear query model, the algorithm can query any vector $x \in \mathbb{R}^{\binom{n}{2}}$ and receives the answer $\langle w, x \rangle$, the inner product of w and x. Linear queries can be much more powerful than cut queries as one can completely learn an unweighted graph with a single linear query. By an information theoretic argument learning an unweighted graph can require $\Omega(n^2/\log(n))$ many cut queries since each cut query reveals at most $O(\log(n))$ bits.

We further introduce a lower bound technique which is a generalization of the cut dimension that we call the ℓ_1-approximate cut dimension. This technique looks not just at mincuts in the graph, but all cuts. We again look at the span of the dimension of these cuts with an additional twist. Suppose the weight of a minimum cut in G is λ and cut S has $w(S) = \lambda + \delta$. Abusing notation we will let S represent both a set of edges and the characteristic vector $S \in \{0,1\}^{\binom{n}{2}}$ of S among all edge slots. The vector S can be *perturbed* to $S-u$ for any vector $u \geq 0$ with $\|u\|_{1,w} \leq \delta$. Here $\|u\|_{1,w} = \sum_i |w(i) \cdot u(i)|$ is the ℓ_1 norm of u weighted by the edge weights of the graph. The ℓ_1-approximate cut dimension of G is then the minimum over all valid perturbations of the dimension of the span of the perturbed cut vectors.

The minimization over all perturbations makes the ℓ_1-approximate cut dimension a difficult quantity to lower bound. We are able to show, however, that

the ℓ_1-approximate cut dimension can be strictly larger than the cut dimension. For every $k \in \mathbb{N}$ and $n = 3k + 1$, we construct an unweighted n-vertex graph G whose ℓ_1-approximate cut dimension is $2n - 2$.

This has the following application.

Theorem 3. *Any deterministic linear query algorithm that correctly computes the minimum cut of all n-vertex weighted undirected graphs must make at least $2n - 2$ queries in the worst case.*

Computing the minimum cut of a graph with cut queries is a special case of finding the nontrivial minimum of a symmetric submodular function $f : 2^V \to \mathbb{R}$ with evaluation queries. That is, to find $\min_{X:\emptyset \neq X \subsetneq V} f(S)$ for a submodular f that satisfies $f(X) = f(V \setminus X)$ for all $X \subseteq V$. As linear queries are more powerful than cut queries, Theorem 3 also implies a $2n - 2$ evaluation query lower bound for a deterministic algorithm finding a nontrivial minimum of a symmetric submodular function, which is currently the best known.

2 Preliminaries

For every natural number n, we denote by $[n]$ the set $\{1, 2, \ldots, n\}$. For a vector $z \in \mathbb{R}^n$ we write $z \geq \mathbf{0}$ if every coordinate of the vector is at least 0, and similarly we write $z = \mathbf{0}$ if z is the all-zero vector. We denote the scalar product of two vectors $z, z' \in \mathbb{R}^n$ by $\langle z, z' \rangle$. For any matrix, denote the rank of A by $\mathrm{rk}(A)$. We denote the disjoint union of sets X and Y by $X \sqcup Y$.

2.1 Graphs, cuts, sets

An undirected *weighted graph* on n vertices is a couple $G = (V, w)$, where V is the set of vertices with $|V| = n$, the set of edge slots $V^{(2)}$ is the set of subsets of V with cardinality 2, and the weight function $w : V^{(2)} \to \mathbb{R}$ is non-negative. We refer to the vertex set of G as $V(G)$. The set of edges of G is defined as $E = \{e \in V^{(2)} : w(e) > 0\}$. When in a graph $G = (V, w)$ the weight of every edge is 1, we say that the graph is *unweighted*, and we refer to it also as $G = (V, E)$; such graph is also called a *simple graph*. For an edge $e = \{u, v\}$, we say that u and v are the endpoints of e. For a subset $X \subseteq V$ of the vertices, we denote by $E(X)$ the set of edges in E which have both endpoints in X, and for disjoint subsets $X, Y \subseteq V$, we denote by $E(X, Y)$ the set of edges with exactly one endpoint in each of the two sets. We extend the weight function w to any subset E' of the edges by $w(E') = \sum_{e \in E'} w(e)$. We will deal only with graphs which have at least 2 vertices.

We fix an ordering $v_1 < v_2 < \cdots < v_n$ of the vertices which induces also an ordering $\{v_1, v_2\}, \{v_1, v_3\}, \ldots, \{v_{n-1}, v_n\}$ of the edge slots as well as an ordering $e_1 < e_2 < \ldots < e_m$ of the $m = |E|$ edges. We view $w \in \mathbb{R}^{\binom{n}{2}}$ as a vector whose i^{th} coordinate gives the (possibly zero) weight of the ith edge slot according to this ordering, and we define $\vec{w} \in \{0, 1\}^m$ as the restriction of w to the edges. With some slight abuse of notation, for a set of edges $S \subseteq E$, we use the same symbol S to also denote the characteristic vector in $\{0, 1\}^{\binom{n}{2}}$ of S among all edge slots. We further need the characteristic vector of $S \subseteq E$ among the m edges E, for which we use the notation $\chi(S) \in \{0, 1\}^m$. For a family $\mathcal{F}$ of subsets of the edges, we use the notation $\vec{\mathcal{F}} = \{\chi(S) \in \{0, 1\}^m : S \in \mathcal{F}\}$.

For $X \subseteq V$, we denote by $\bar{X}$ the set $V \setminus X$. A *cut* S is a set $E(X, \bar{X})$ for some $\emptyset \neq X \subsetneq V$. We call X and $\bar{X}$ the *shores* of S, and we denote the cut by $\Delta(X)$. A cut is a *star cut* if one of its shores is a singleton, otherwise it is *non-star* cut. If the singleton shore of a star cut S is $\{v\}$, then we say that S is a star cut at v. The *weight* of a cut is the sum of the weights of its edges. For a cut S we define the *graph of the cut* S as the unweighted graph $G(S) = (V', E')$ where V' is the set of vertices in V that are endpoints of at least one edge in S, and $E' = S$. We say that a cut S is *connected* if $G(S)$ is a connected graph. A cut is a *minimum* cut, or mincut, for short, if no other cut has smaller weight. We denote by $\mathcal{M}(G)$ be the set of minimum cuts of G. The *cut dimension* of G is $\text{cdim}(G) = \dim(\text{span}(\vec{\mathcal{M}}(G)))$.

Let V be a set of size n. Two sets $X, Y \subseteq V$ are said to *overlap* if $X \cap Y \neq \emptyset, \bar{X} \cap Y \neq \emptyset, X \cap \bar{Y} \neq \emptyset$. A family $\mathcal{G}$ of subsets of V is said to be *laminar* if for all $X, Y \in \mathcal{G}$ it holds that X and Y do not overlap. A set family $\mathcal{G} \subseteq 2^V$ is said to be *closed under overlaps* if for every $X, Y \in \mathcal{G}$ that overlap it holds that $X \cap Y, X \cup Y \in \mathcal{G}$. A laminar subset $\mathcal{L} \subseteq \mathcal{G}$ is said to be *maximal in* $\mathcal{G}$ if for every $X \in \mathcal{G} - \mathcal{L}$ there is a $Y \in \mathcal{L}$ such that X, Y overlap. We say a laminar subset $\mathcal{L}$ is maximal if it is maximal in 2^V.

The sets $X, Y \subseteq V$ *cross* if they overlap and additionally $\bar{X} \cap \bar{Y} \neq \emptyset$. Note that if X, Y cross then so do $X, \bar{Y}$. A set family $\mathcal{G} \subseteq 2^V$ is said to be *cross-free* if for all $X, Y \in \mathcal{G}$ it holds that X and Y do not cross. Observe that if X and Y do not cross then either Y or $\bar{Y}$ is a subset of X or $\bar{X}$. Let $G = (V, w)$ be a graph with n vertices. Two cuts $\Delta(X)$ and $\Delta(Y)$ of G are *crossing* if X and Y are crossing. Let $\mathcal{F} = \{\Delta(X_1), \ldots, \Delta(X_k)\}$ be a set of cuts of G. We say that $\mathcal{F}$ is *cross-free family of cuts* if $\mathcal{G} = \{X_1, \ldots, X_k\}$ is cross-free. Note that it does not matter which shore we take to be in $\mathcal{G}$.

There is a close relationship between cross-free families of cuts and laminar sets. Let $\mathcal{F} = \{\Delta(X_1), \ldots, \Delta(X_k)\}$ be a cross-free family of cuts where each $X_i \subseteq V$, and let $X'_i = X_i$ if $v_1 \notin X_i$ and $X'_i = \bar{X}_i$ otherwise. The *beach* of $\mathcal{F}$ is the set $\mathcal{G} = \{X'_1, \ldots, X'_k\}$. For a family of sets $\mathcal{G} \subseteq 2^V$ we say that it is *proper* if $\emptyset, V \notin \mathcal{G}$, and we say that it is *complement free* if it does not contain X, Y with $Y = \bar{X}$.

Claim 4. *Let $\mathcal{F}$ be a cross-free family of distinct cuts and $\mathcal{G}$ its beach. Then $\mathcal{G}$ is proper, complement free and laminar.*

Proof. First, $\mathcal{G}$ does not contain $\emptyset$ or V because these are not shores of cuts. It is complement free because $\mathcal{F}$ contains distinct cuts, and its beach contains exactly one representative shore from each cut. Finally, we show that it is laminar. Let $X_1, X_2 \in \mathcal{G}$. By definition of a beach, neither of these sets contain v_1, thus $\bar{X}_1 \cap \bar{X}_2 \neq \emptyset$. Therefore if X_1, X_2 overlapped they would also cross, in contradiction to $\mathcal{F}$ being a cross-free family of cuts. □

A mincut is *crossless* if no other mincut crosses it. Observe that a star mincut is always crossless. Also, if a mincut $\Delta(X)$ is crossless then for every mincut $\Delta(Y)$, either Y or $\bar{Y}$ is a subset of X or $\bar{X}$. Crossing mincuts have a nice structural property which was already observed by [DKL76].

Claim 5. *Let $G = (V, w)$ be a weighted graph. If $\Delta(X), \Delta(Y) \in \mathcal{M}(G)$ cross then $\Delta(X \cap Y), \Delta(X \cup Y) \in \mathcal{M}(G)$.*

Proof. We have $\Delta(X \cap Y) \neq \emptyset$ and $\Delta(X \cup Y) \neq V$ because $\Delta(X)$ and $\Delta(Y)$ cross. The cut function is submodular therefore we have

$$w(\Delta(X \cap Y)) + w(\Delta(X \cup Y)) \leq w(\Delta(X)) + w(\Delta(Y)).$$

Let c be the weight of a minimum cut in G. Then the right hand side of the above inequality is equal to $2c$, while its left hand side is at least $2c$. Therefore $w(\Delta(X \cap Y)) + w(\Delta(X \cup Y)) = 2c$ from which the statement follows. □

2.2 Operations on graphs

We will use several operations on graphs. The first of these is the *direct union*.

Definition 6 (direct union)**.** For two graphs $G_0 = (V_0, w_0), G_1 = (V_1, w_1)$ with disjoint vertex sets, and for vertices $v_0 \in V_0$ and $v_1 \in V_1$, the *direct union* of G_0 and G_1 at vertices v_0, v_1 is the fusion of the two by identifying v_0 and v_1. Formally, the direct union is $G_0^{v_0} \oplus G_1^{v_1} = (V, w)$ where $V = (V_0 \cup V_1 \cup \{v\}) \setminus \{v_0, v_1\}$, for a new vertex $v \notin V_0 \cup V_1$. The weight function of $G_0^{v_0} \oplus G_1^{v_1}$ is defined by

$$w(\{x,y\}) = \begin{cases} w_b(\{x,y\}) & \text{if } x,y \in V_b \setminus \{v_b\}, b \in \{0,1\}, \\ w_b(\{x,v_b\}) & \text{if } x \in V_b \setminus \{v_b\}, y = v, b \in \{0,1\}, \\ 0 & \text{otherwise .} \end{cases}$$

The cut dimension of a direct union is a simple function of the cut dimensions of its components.

Claim 7. *Let $G = G_0^{v_0} \oplus G_1^{v_1}$ be the direct union of G_0 and G_1 at vertices v_0, v_1. Let c_b be the weight of a minimum cut in G_b, for $b = 0, 1$. Then* $\operatorname{cdim}(G) = \operatorname{cdim}(G_0) + \operatorname{cdim}(G_1)$ *if $c_0 = c_1$, and* $\operatorname{cdim}(G) = \operatorname{cdim}(G_b)$ *if $c_b < c_{1-b}$.*

Proof. Let $\Delta(X)$ be an arbitrary cut of G where $v \notin X$. If $X \not\subseteq V_b$, for $b \in \{0,1\}$, then the weight of the cut $\Delta(X)$ is at least $c_0 + c_1$, and therefore it is not a minimum cut. If $X \subseteq V_b$, for some $b \in \{0,1\}$ then the weight of $\Delta(X)$ in G is the same as the weight of $\Delta(X)$ in G_b. Therefore if $c_0 = c_1$ then every mincut in G_0 and every mincut of G_1 is a mincut of G, and these are the only mincuts. Since their supports are disjoint, we have $\operatorname{cdim}(G) = \operatorname{cdim}(G_0) + \operatorname{cdim}(G_1)$. If $c_b < c_{1-b}$ then only the mincuts of G_b are mincuts of G, and therefore $\operatorname{cdim}(G) = \operatorname{cdim}(G_b)$. □

The next two operations, which are inverses of each other, give a decomposition of a graph along a cut into two smaller graphs, and a composition of two graphs into a bigger one by unfolding a star cut in each components. The decomposition operation was essentially defined in [FF09]. Let $G = (V, w)$ be a weighted graph and let Z be a cut in G with shores X_0 and $X_1 = V \setminus X_0$. The *separation* of G along the cut Z, denoted by $\operatorname{sep}(G, Z)$, is the set of two graphs $\{G_0 = (V_0, w_0), G_1 = (V_1, w_1)\}$, where $V_b = X_b \cup \{v_{1-b}\}$, for $b = 0, 1$ with new vertices v_0, v_1. The respective weight functions are defined by $w_b(\{x,y\}) = w(\{x,y\})$ for any $x, y \in X_b$, and $w_b(\{x, v_{1-b}\}) = \sum_{y \in V_{1-b}} w(\{x,y\})$ for any $x \in X_b$.

Let $G_0 = (V_0, w_0), G_1 = (V_1, w_1)$ be two graphs on disjoint vertex sets, and let $v_b \in V_{1-b}$ be arbitrary vertices for $b \in \{0,1\}$. The *merge* of G_0 and

G_1 along the vertices v_1, v_0, denoted by $\mathrm{mer}(\{(G_0, v_1), (G_1, v_0)\})$, is the graph $G = (V, w)$, where $V = (V_0 \cup V_1) \setminus \{v_0, v_1\}$. The weight function in G is defined by $w(\{x, y\}) = w_b(\{x, y\})$ if $x, y \in V_b$, for $b \in \{0, 1\}$, and

$$w(\{x, y\}) = w_0(\{x, v_1\})w_1(\{v_0, y\}), \text{ if } x \in V_0 \text{ and } y \in V_1.$$

It follows from the definitions sep is the left inverse of mer if the star cut at v_1 in V_0 and the star cut at v_0 in V_1 both have weight one, and sep is the right inverse of mer if the weight of the cut Z is one. We formally state the former property.

Claim 8. *Let $G_0 = (V_0, w_0)$ and $G_1 = (V_1, w_1)$ have disjoint vertex sets, and let $v_b \in V_{1-b}$ such that $w_b(\Delta(v_{1-b})) = 1$, for $b = 0, 1$. Let Z be the cut in $\mathrm{mer}(\{(G_0, v_1), (G_1, v_0)\})$ whose shores are $V_0 \setminus \{v_1\}$ and $V_1 \setminus \{v_0\}$. Then $w(Z) = 1$ and*

$$\mathrm{sep}(\mathrm{mer}(\{(G_0, v_1), (G_1, v_0)\}), Z) = \{G_0, G_1\}.$$

2.3 Query models

Definition 9 (MINCUT_n)**.** The input in the MINCUT_n problem is an n-vertex weighted undirected graph $G = (V, w)$. The required output on G is the weight of a minimum cut in G.

A deterministic algorithm correctly solves the MINCUT_n problem if it outputs the correct mincut weight for every n-vertex input graph G. We consider algorithms given two models of query access to the input graph $G = (V, w)$, linear queries and cut queries. A *linear query* for G is a vector $x \in \mathbb{R}^{\binom{n}{2}}$, and the query is answered by $\langle x, w \rangle$. A cut query is a vector $x \in \{0, 1\}^{\binom{n}{2}}$ which is the characteristic vector of a cut in the complete n-vertex graph. The answer to a cut query is again $\langle x, w \rangle$. Clearly any cut query algorithm can be simulated by a linear query algorithm.

We use $D_{\mathrm{cut}}(\mathrm{MINCUT}_n)$ to denote the minimum, over all deterministic query algorithms $\mathcal{A}$ that correctly solve MINCUT_n, of the maximum over all n-vertex input graphs $G = (V, w)$ of the number of cut queries made by $\mathcal{A}$ on G. $D_{\mathrm{lin}}(\mathrm{MINCUT}_n)$ is defined analogously for linear queries.

Some authors instead define the output of the minimum cut problem to be a cut S that achieves the minimum weight, rather than the weight itself. Over n-vertex weighted graphs let us denote this problem as $\mathrm{ARGMINCUT}_n$. For linear and cut queries, an algorithm that finds a minimum cut S can also return the weight of S with one additional query. Thus $D_{\mathrm{lin,cut}}(\mathrm{ARGMINCUT}_n) \geq D_{\mathrm{lin,cut}}(\mathrm{MINCUT}_n) - 1$, and the lower bounds we prove for MINCUT_n can be applied, minus 1, to $\mathrm{ARGMINCUT}_n$ as well.

3 Lower bounds on the linear query complexity of MINCUT

Graur et al. [GPRW20] introduce the cut dimension as a means to show lower bounds on the deterministic cut query complexity of computing minimum cut.

Theorem 10 ([GPRW20]). *If there is an n-vertex weighted graph $G = (V, w)$ with $\mathrm{cdim}(G) = k$ then $D_{\mathrm{cut}}(\mathrm{MINCUT}_n) \geq k$.*

We show that this theorem even holds with respect to a stronger computational model where the algorithm is able to make linear queries. We also give a generalization of the cut dimension to a quantity which is at least as large, and can be strictly larger, that we call the ℓ_1-approximate cut dimension. We now give an overview of the Graur et al. [GPRW20] argument in the context of linear queries and how we can extend it.

The proof of Theorem 10 is based on an adversary argument. Suppose a deterministic cut query algorithm makes k linear queries and consider the execution of the algorithm on a fixed n-vertex graph $G = (V, w)$ whose set of minimum cuts is $\mathcal{M}(G)$. Make a k-by-$\binom{n}{2}$ matrix A whose rows are the query vectors asked by the algorithm. Suppose we can find a vector $z \in \mathbb{R}^{\binom{n}{2}}$ such that

1. $w - z \geq \mathbf{0}$,
2. $Az = \mathbf{0}$,
3. There is a cut $S \in \mathcal{M}(G)$ such that $\langle S, z\rangle > 0$.

The existence of such a vector z means the algorithm cannot correctly compute minimum cut weight on all weighted n-vertex graphs. The reason is that $G' = (V, w - z)$ is a valid non-negatively weighted graph by (1), has the same answers on all queries asked by the algorithm by (2), and by (3) has minimum cut weight at most $\langle S, w - z\rangle = \langle S, w\rangle - \langle S, z\rangle < \langle S, w\rangle$, which is strictly less than the minimum cut weight of G. As with k queries the algorithm cannot distinguish whether the input is G or G', it cannot correctly output the minimum cut weight for all n-vertex weighted graphs.

A weaker condition than (3) suffices for this argument to work. Suppose that the minimum cut weight in G is c^*. Then the argument still goes through with the condition

3'. There is a cut S such that $\langle S, z\rangle > \langle S, w\rangle - c^*$.

This is because the algorithm cannot distinguish the graph G with minimum cut weight c^* from the graph $G' = (V, w - z)$ which has minimum cut weight at most $\langle S, w - z\rangle < c^*$.

In order to understand what kind of bound this argument gives, for fixed w, A, S we define the quantity $\alpha(w, A, S)$ which is given by the following linear program.

$$\begin{aligned} \alpha(w, A, S) = \underset{z}{\text{maximize}} \quad & \langle S, z\rangle \\ \text{subject to} \quad & w - z \geq 0 \\ & Az = \mathbf{0} \end{aligned}$$

Taking the dual of this program gives

$$\begin{aligned} \alpha(w, A, S) = \underset{v}{\text{minimize}} \quad & \langle S - A^T v, w\rangle \\ \text{subject to} \quad & S - A^T v \geq 0 \end{aligned}$$

The dual tells us that a vector z having large overlap with S and satisfying items $(1), (2)$ above exists iff the vector S is *far away* from the rowspace of A. The notion of far away here is a one-sided ℓ_1 distance weighted by w. It is one-sided because the condition $S - A^T v \geq 0$ tells us we are looking to approximate S by vectors in the rowspace of A that are entrywise at most S. As $S - A^T v \geq 0$ and $w \geq 0$ this means $\langle S - A^T v, w \rangle = \sum_i |w(i) \cdot (S(i) - A^T v)| = \|S - A^T v\|_{1,w}$, where $\|u\|_{1,w}$ is defined to be $\sum_i |u(i)w(i)|$. Thus the value of the dual can be interpreted as the one-sided $\|\cdot\|_{1,w}$ distance between S and the rowspace of A.

This leads us to define an ℓ_1 approximate version of the cut dimension. The notion we need is given by the following definitions.

Definition 11 (one-sided row-by-row ℓ_1-approximate rank)**.** Let $Y \in \mathbb{R}^{M \times N}$ be a matrix, $w \in \mathbb{R}^N$ a weight vector and $c \in \mathbb{R}^M$ a cost vector. We define the (w, c) one-sided row-by-row ℓ_1-approximate rank of Y to be the minimum rank of a matrix $\tilde{Y}$ such that $\tilde{Y} \leq Y$ and $\|Y(i,:) - \tilde{Y}(i,:)\|_{1,w} \leq c(i)$, for all $1 \leq i \leq M$.

Definition 12 (ℓ_1-approximate cut dimension)**.** Let $G = (V, w)$ be an n-vertex weighted undirected graph with minimum cut weight c^*. Let M be $(2^{n-1}-1)$-by-$\binom{n}{2}$ matrix whose rows are $S \in \{0,1\}^{\binom{n}{2}}$ for all cuts S of G. Let $c = Mw - c^*\mathbf{1}$, where $\mathbf{1}$ is the all one vector. Then the ℓ_1-approximate cut dimension of G, denoted $\widetilde{\mathrm{cdim}}(G)$, is the (w, c) one-sided row-by-row ℓ_1-approximate rank of M.

Theorem 13. *If there is an n-vertex graph weighted graph $G = (V, w)$ with $\widetilde{\mathrm{cdim}}(G) = k$ then $D_{\mathrm{lin}}(\mathrm{MINCUT}_n) \geq k$.*

Proof. Let $G = (V, w)$ be a graph with $\widetilde{\mathrm{cdim}}(G) = k$ and let c^* be the minimum cut weight of G. Suppose for contradiction there is a deterministic $k-1$ linear query algorithm that correctly computes the minimum cut of any n-vertex graph. Run this algorithm answering queries according to G and package the queries into a $(k-1)$-by-$\binom{n}{2}$ matrix A.

As the algorithm is correct, for every cut S of G it must be the case that $\alpha(w, A, S) \leq \langle S, w \rangle - c^*$. If not, the graph $G' = (V, w - z)$, where z is an optimal solution to the primal of $\alpha(w, A, S)$, has minimum cut weight strictly smaller than c^*, yet G' cannot be distinguished from G by the algorithm. Thus by the dual formulation of $\alpha(w, A, S)$, this means that for every cut S of G there is a vector $\tilde{S} = A^T v$ in the rowspace of A such that $\tilde{S} \leq S$ and $\|S - \tilde{S}\|_{1,w} \leq \langle S, w \rangle - c^*$. The matrix $\tilde{M}$ whose rows are $\tilde{S}$ for all cuts S therefore witnesses that $\widetilde{\mathrm{cdim}}(G) \leq \mathrm{rk}(A) \leq k - 1$, a contradiction. □

Lemma 14. *For any weighted graph $G = (V, w)$ we have $\mathrm{cdim}(G) \leq \widetilde{\mathrm{cdim}}(G)$.*

Proof. Suppose that $G = (V, w)$ has minimum cut weight c^*, and let $\mathcal{M}(G)$ be the set of minimum cuts of G. Let M be the $(2^{n-1} - 1)$-by-$\binom{n}{2}$ matrix whose rows are $S \in \{0,1\}^{\binom{n}{2}}$ for all cuts S of G and let $c = Mw - c^*$.

Let Y be the submatrix of M where rows are restricted to cuts in $\mathcal{M}(G)$ and columns are restricted to the edge slots e where $w(e) > 0$. Thus the rows of Y are exactly the vectors $\chi(S)$ for $S \in \mathcal{M}(G)$. and the rank of Y is $\mathrm{cdim}(G)$. Any matrix $\tilde{M}$ which satisfies $\tilde{M} \leq M$ and $\|M(i,:) - \tilde{M}(i,:)\|_{1,w} \leq c(i)$ for all i must contain Y as a submatrix, as $c(i) = 0$ for rows i that correspond to minimum cuts and w is positive on the edge slots labeling the columns of Y.

Thus $\mathrm{rk}(\tilde{M}) \geq \mathrm{rk}(Y)$ for any (w, c) one-sided row-by-row ℓ_1 approximation $\tilde{M}$ of M, giving the lemma. □

In Section 7 we will see that $\widetilde{\mathrm{cdim}}(G)$ can be strictly larger than $\mathrm{cdim}(G)$. From Theorem 13 and Lemma 14 we obtain the following corollary.

Corollary 15. *If there is an n-vertex weighted graph $G = (V, w)$ with $\mathrm{cdim}(G) = k$ then $D_{\mathrm{lin}}(\mathrm{MINCUT}_n) \geq k$.*

7 ℓ_1-approximate cut dimension

In this section, we use the ℓ_1-approximate cut dimension method to show Theorem 3 that for any $k \in \mathbb{N}$ and $n = 3k+1$, it holds that $D_{\mathrm{lin}}(\mathrm{MINCUT}_n) \geq 2n-2$.

Let K_4 be the complete graph on 4 vertices with all edge weights equal to 1. The theorem will follow from showing that the ℓ_1-approximate cut dimension of the direct union of k copies of K_4 has ℓ_1-approximate cut dimension $6k$. We start with the base case $k = 1$ to build up the notation and intuition that will be needed for the general case. The following definition and fact will be useful.

Definition 37 (Strictly diagonally dominant)**.** Let $A \in \mathbb{R}^{n \times n}$ be a matrix. We say that the ith row of A is *strictly diagonally dominant* if $|A(i,i)| > \sum_{j \neq i} |A(i,j)|$. We say that A is strictly diagonally dominant iff all of its rows are.

It is well known that a strictly diagonally dominant matrix has full rank. One way to prove this is via the following fact, which we will make use of in the proof of Theorem 3.

Fact 38. *Let $A \in \mathbb{R}^{n \times n}$ be a matrix whose i^{th} row is strictly diagonally dominant. If $Au = \mathbf{0}$ for a vector $u \neq \mathbf{0}$ then $|u_i| < \|u\|_\infty$.*

Proof. Suppose for a contradiction that for some $u \neq \mathbf{0}$ it holds that $Au = \mathbf{0}$ and $|u_i| = \|u\|_\infty$ where the ith row of A is strictly diagonally dominant. By normalizing and flipping the sign of u if necessary we may assume $\|u\|_\infty = 1$ and $A(i,i)u_i = |A(i,i)|$. Thus

$$\sum_j A(i,j)u_j = |A(i,i)| + \sum_{j \neq i} A(i,j)u_j \geq |A(i,i)| - \sum_{j \neq i} |A(i,j)| > 0 \ ,$$

a contradiction. □

7.1 ℓ_1-approximate cut dimension of K_4

We label the vertices of K_4 by a, b, c, v, and use the ordering of edges indicated in Fig. 1. Let X be the 7-by-6 matrix whose rows correspond to the cut vectors of all the nontrivial cuts

$$X = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 & 1 \end{bmatrix} . \tag{1}$$

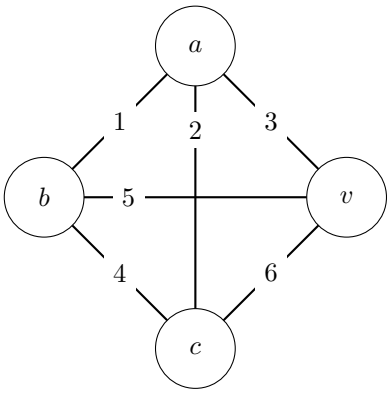


Figure 1: The complete graph on 4 vertices with all edge weights equal to 1. The labels on edges indicate the ordering of edges used to represent cut vectors in the proof.

The cut vectors in X are given in the order

$$\Delta(\{a\}), \Delta(\{b\}), \Delta(\{c\}), \Delta(\{a,b,c\}), \Delta(\{a,b\}), \Delta(\{a,c\}), \Delta(\{b,c\}) \ .$$

The first 4 rows correspond to star cuts which are minimum cuts of weight 3 in K_4. The last three rows correspond to cuts which have weight 4 in K_4. Thus to show a lower bound of 6 on the number of linear queries needed to compute the minimum cut of a 4 vertex graph, we need to show that the $w = (1,1,1,1,1,1), c = (0,0,0,0,1,1,1)$ one-sided ℓ_1 approximate rank of X is 6.

Claim 39. *Let $w = \mathbf{1} \in \mathbb{R}^6$, and $c = (0,0,0,0,1,1,1)$. The (w,c) one-sided ℓ_1 approximate rank of X is 6.*

Proof. The rank of X at most 6 as this is the number of columns, which takes care of the upper bound.

Now consider the lower bound. To do this we need to lower bound the rank of the matrix

$$Z = X - \begin{bmatrix} \mathbf{0}_{4,2} & \mathbf{0}_{4,2} & \mathbf{0}_{4,2} \\ A_1 & A_2 & A_3 \end{bmatrix}$$

where each of $A_1, A_2, A_3 \geq 0$ are 3-by-2 matrices and every row of $A_1 + A_2 + A_3$ sums to at most 1. As the first 4 rows of X correspond to vectors of minimum cuts, no error is allowed on the first 4 rows.

The first 4 rows of Z are equal to the first 4 rows of X, as there is no perturbation allowed on these rows. By doing elementary row operations on the first four rows, which do not change the rank, we can transform the first four rows of Z into the reduced row echelon form of $X(1:4,:)$. Thus we arrive at the following matrix.

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & -1 \\ 0 & 1 & 0 & 0 & -1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 & 1 \end{bmatrix} - \begin{bmatrix} \mathbf{0}_{4,2} & \mathbf{0}_{4,2} & \mathbf{0}_{4,2} \\ A_1 & A_2 & A_3 \end{bmatrix} \ .$$

Now we do column operations to zero out the entries in the first four rows and last two columns. For a m-by-2 matrix A we will use the notation A° to denote the matrix A with the order of the columns swapped. We arrive at

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & -2 \\ 1 & 0 & 1 & 1 & -2 & 0 \\ 1 & 1 & 0 & 0 & 2 & 2 \end{bmatrix} - \begin{bmatrix} \mathbf{0}_{4,2} & \mathbf{0}_{4,2} & \mathbf{0}_{4,2} \\ A_1 & A_2 & A_1^\circ - A_2 - A_2^\circ + A_3 \end{bmatrix}.$$

Finally, we can do row operations to zero out the first four columns in the last three rows.

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -2 \\ 0 & 0 & 0 & 0 & -2 & 0 \\ 0 & 0 & 0 & 0 & 2 & 2 \end{bmatrix} - \begin{bmatrix} \mathbf{0}_{4,2} & \mathbf{0}_{4,2} & \mathbf{0}_{4,2} \\ \mathbf{0}_{3,2} & \mathbf{0}_{3,2} & A_1^\circ - A_2 - A_2^\circ + A_3 \end{bmatrix}.$$

The task has now reduced to showing the matrix

$$Z' = \begin{bmatrix} 2 & 0 \\ 0 & 2 \\ -2 & -2 \end{bmatrix} + A_1 - A_2 - A_2^\circ + A_3^\circ$$

has rank 2 for any A_1, A_2, A_3 satisfying the constraints. Let us simplify the matrix $A_1 - A_2 - A_2^\circ + A_3^\circ$. First, let $A_1' = A_1 + A_3^\circ$. Next, note that $D = A_2 + A_2^\circ$ has the property that $D(i,1) = D(i,2)$ for $i \in [3]$. In the sequel we call this the *partner property*.

As the row sum of $A_1' + A_2$ is at most 1, unless $A_1'(1:2, 1:2) = \mathbf{0}_{2,2}$ and at least one row sum of $A_2(1:2, 1:2)$ is equal to 1 the first two rows of Z' will be strictly diagonally dominant. If the first two rows of Z' are strictly diagonally dominant then the rank of Z' must be 2, thus we now handle the "unless" case.

First, suppose exactly one row sum of $A_2(1:2, 1:2)$ is equal to 1. Say without loss of generality it is the second one, thus the first row of Z' is strictly diagonally dominant. Then for a sufficiently small ε we can multiply the first column by $1 - \varepsilon$ so that the first row remains strictly diagonally dominant and the second row becomes strictly diagonally dominant as well. This does not increase the rank and thus shows again that the rank of Z' is 2.

The remaining case is where both rows of $A_2(1:2, 1:2)$ sum to one. In this case by the partner property we have

$$Z'(1:2, 1:2) = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}.$$

On the other hand, the last row of Z' must have both entries ≤ -1. Thus the determinant of the submatrix formed by the first row and the third is strictly negative and so Z' has rank 2. $\square$

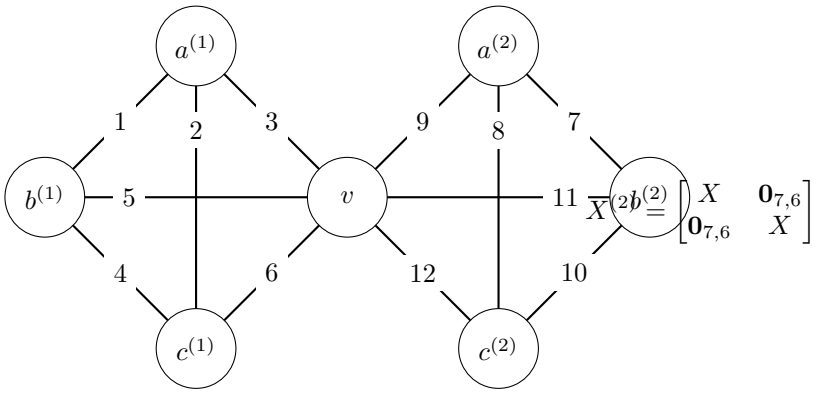


Figure 2: Example of the direct union of two copies of K_4. With the ordering of the edges given by the edge labels, the matrix of cut vectors of the cuts $\Delta(\{a^{(i)}\}), \Delta(\{b^{(i)}\}), \Delta(\{c^{(i)}\}), \Delta(\{a^{(i)}, b^{(i)}, c^{(i)}\}), \Delta(\{a^{(i)}, b^{(i)}\}), \Delta(\{a^{(i)}, c^{(i)}\}), \Delta(\{b^{(i)}, c^{(i)}\})$ for $i \in [2]$ becomes the matrix $X^{(2)}$ on the right.

7.2 Direct union of K_4 with itself

Now we prove the general case. The key to the proof is the following lemma.

Lemma 40. *Let $k \in \mathbb{N}$ and B be the $3k$-by-$2k$ matrix*

$$B = \begin{bmatrix} 2\mathbf{I}_{2k} \\ -2\mathbf{I}_k \otimes [1,1] \end{bmatrix} .$$

For any matrices $3k$-by-$2k$ matrices A_1, A_2 satisfying the conditions

1. *$A_1, A_2 \geq 0$*
2. *(partner property) For all $i \in [3k]$ and $j \in [k]$ it holds that $A_2(i, 2j-1) = A_2(i, 2j)$.*
3. *Every row of $A_1 + A_2/2$ sums to at most 1*

it holds that $B + A_1 - A_2$ has rank $2k$.

Proof. The rank is at most $2k$ as that is the number of columns; we focus on showing the columns are linearly independent.

Let $Z = B + A_1 - A_2$. We call the first $2k$ rows of Z rows of type I, and the last k rows of type II. If a type I row is not strictly diagonally dominant, we call it *full*. Notice that a type I row i is full if and only if the ith row of A_1 is zero and the ith row of A_2 sums to 2. In this case, $Z(i,j) \leq 0$ for every $j \neq i$ and it holds that $Z(i,i) = -\sum_{j \neq i} Z(i,j)$. For $i \in [k]$ we call $2i-1$ and $2i$ *partners*.

Suppose for contradiction there is a vector $\vec{u} \neq \mathbf{0}$ such that $A\vec{u} = \mathbf{0}$. As $\vec{u} \neq \mathbf{0}$ by normalizing and multiplying by -1 as needed we may assume that $\|u\|_\infty = 1$ and i is a coordinate with $\vec{u}(i) = 1$. By Fact 38 the ith row of Z, which is a type I row, cannot be strictly diagonally dominant. Thus the ith row must be full. Therefore for $Z(i,:)\vec{u} = 0$ to hold it must be the case that $\vec{u}(j) = 1$ for every j where $A_2(i,j) > 0$. Such a j must exist as the ith row of A_2 sums to 2. So let j be a coordinate with $A_2(i,j) > 0$ and let j' be the partner of j. By the partner property we also have $A_2(i,j') > 0$ and therefore $\vec{u}(j) = \vec{u}(j') = 1$.

Now consider the type II row ℓ for which $B(\ell, j) = B(\ell, j') = -2$. As $B(\ell, t) = 0$ for $t \notin \{j, j'\}$ this means

$$\begin{aligned} Z(\ell, :)\vec{u} &= Z(\ell, j) + Z(\ell, j') + \sum_{t \notin \{j,j'\}} Z(\ell, t)\vec{u}(t) \\ &\leq B(\ell, j) + A_1(\ell, j) + B(\ell, j') + A_1(\ell, j') + \|\vec{u}\|_\infty \sum_{t \notin \{j,j'\}} |Z(\ell, t)| \\ &\leq -4 + \sum_t A_1(\ell, t) + A_2(\ell, t) \\ &\leq -2 \ , \end{aligned}$$

and we have arrived at a contradiction. □

With Lemma 40 in hand we are now ready to prove Theorem 3.

Proof of Theorem 3. Let $G^{(1)}, \ldots, G^{(k)}$ be k copies of K_4 where the vertices in $G^{(i)}$ are labeled by $a^{(i)}, b^{(i)}, c^{(i)}, v^{(i)}$ for $i \in [k]$. The graph G is formed by taking the direct union of $G^{(1)}, \ldots, G^{(k)}$ at the vertices $v^{(1)}, \ldots, v^{(k)}$. That is, the vertices $v^{(1)}, \ldots, v^{(k)}$ are all identified by a common vertex denoted v. See Fig. 2 for an illustration of the graph for $k = 2$.

The cuts of G we focus on are the $7k$ cuts given by

$$\Delta(\{a^{(i)}\}), \Delta(\{b^{(i)}\}), \Delta(\{c^{(i)}\}), \Delta(\{a^{(i)}, b^{(i)}, c^{(i)}\}), \Delta(\{a^{(i)}, b^{(i)}\}), \Delta(\{a^{(i)}, c^{(i)}\}), \Delta(\{b^{(i)}, c^{(i)}\} \ ,$$

for $i \in [k]$. For any $i \in [k]$ the cuts $\Delta(\{a^{(i)}\}), \Delta(\{b^{(i)}\}), \Delta(\{c^{(i)}\}), \Delta(\{a^{(i)}, b^{(i)}, c^{(i)}\})$ achieve the minimum cut weight of G, which is 3, and the cuts $\Delta(\{a^{(i)}, b^{(i)}\}), \Delta(\{a^{(i)}, c^{(i)}\}), \Delta(\{b^{(i)}, c^{(i)}\}$ have weight 4.

With an ordering of the edges as exemplified in Fig. 2, the matrix of cut vectors of these cuts is $X^{(k)} = \mathbf{I}_k \otimes X$, where X is the matrix from Eq. (1). In every nonzero block of $X^{(k)}$ the first four rows are minimum cuts with weight 3 and the last 3 rows are cuts with weight 4. Let $c' = (0, 0, 0, 0, 1, 1, 1)$. The theorem will follow from Theorem 13 by showing that the $w = \mathbf{1}_{6k}, c = \mathbf{1}_k \otimes c'$ one-sided ℓ_1 approximate rank of $X^{(k)}$ is $6k$.

To do this, we must show that $X^{(k)} - A$ has rank $6k$ for any matrix $A \geq 0$ which is all zero on any row of $I_k \otimes X$ corresponding to a minimum cut, and where the row sum of A is at most 1 on any row of $I_k \otimes X$ corresponding to a cut of weight 4. In order to make reference to the base case, it will be useful to partition the columns into k blocks of 6 columns, where the ith block is further partitioned into blocks of size 2 represented by the $7k$-by-2 matrices $A_1^{(i)}, A_2^{(i)}, A_3^{(i)}$. In other words, we view A as follows

$$A = \begin{bmatrix} A_1^{(1)} & A_2^{(1)} & A_3^{(1)} & \cdots & A_1^{(k)} & A_2^{(k)} & A_3^{(k)} \end{bmatrix}$$

where each $A_j^{(i)}$ for $j \in [3], i \in [k]$ is a $7k$-by-2 matrix.

As in the base case, we begin by doing Gauss-Jordan elimination on the rows corresponding to mincuts of each X block in $X^{(k)}$. These operations only touch rows corresponding to mincuts where A is zero, thus they do not change

A. After these operations we arrive at the matrix $\mathbf{I}_k \otimes X' - A$ where

$$X' = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & -1 \\ 0 & 1 & 0 & 0 & -1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 & 1 \end{bmatrix}$$

Next, as in the base case, we do column operations to zero out the last two columns in the first four rows of each block of X'. This gives us the matrix $\mathbf{I}_k \otimes X'' - A'$ where

$$X'' = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & -2 \\ 1 & 0 & 1 & 1 & -2 & 0 \\ 1 & 1 & 0 & 0 & 2 & 2 \end{bmatrix}$$

and the ith block of A' looks like

$$[A_1^{(i)} \mid A_2^{(i)} \mid A_1^{(i)\circ} - A_2^{(i)} - A_2^{(i)\circ} + A_3^{(i)}]. \quad .$$

Here $A_1^{(i)\circ}$ denotes the matrix $A_1^{(i)}$ with the order of the columns swapped. Finally, we use $X''(1:4, 1:4)$ to zero out all other entries of $\mathbf{I}_k \otimes X'' - A'$ in the first 4 columns of each block. This brings us to the matrix $\mathbf{I}_k \otimes X''' - A''$ where

$$X''' = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -2 \\ 0 & 0 & 0 & 0 & -2 & 0 \\ 0 & 0 & 0 & 0 & 2 & 2 \end{bmatrix}$$

and the ith block of A'' is

$$[\mathbf{0}_{7k,2} \mid \mathbf{0}_{7k,2} \mid A_1^{(i)\circ} - A_2^{(i)} - A_2^{(i)\circ} + A_3^{(i)}] \quad .$$

Again, each of $A_1^{(i)}, A_2^{(i)}, A_3^{(i)}$ is zero on rows corresponding to minimum cuts. Thus by multiplying the last two columns of each block by -1 and permuting rows and columns we can transform $\mathbf{I}_k \otimes X''' - A''$ into the form

$$\begin{bmatrix} \mathbf{I}_{4k} & \mathbf{0}_{4k,2k} \\ \mathbf{0}_{3k,4k} & B + A_1 - A_2 \end{bmatrix}$$

where B, A_1, A_2 satisfy the conditions of Lemma 40. Thus a rank lower bound of $6k$ follows from the lower bound of $2k$ on the rank of $B + A_1 - A_2$ given in Lemma 40. □

References

[ACK20] Sepehr Assadi, Deeparnab Chakrabarty, and Sanjeev Khanna. Graph connectivity and single element recovery via linear and OR queries. *CoRR*, abs/2007.06098, 2020.

[BFS86] László Babai, Peter Frankl, and Janos Simon. Complexity classes in communication complexity theory (preliminary version). In *27th Annual Symposium on Foundations of Computer Science, Toronto, Canada, 27-29 October 1986*, pages 337–347, 1986.

[BG08] András A. Benczúr and Michel X. Goemans. Deformable polygon representations and near-mincuts. In Martin Grøtschel and Gyula O. H. Katona, editors, *Building Bridges: Between Mathematics and Computer Science*, volume 19 of *Bolyai Society Mathematical Studies*, pages 103–135. Springer, 2008.

[Bix75] R. E. Bixby. The minimum number of edges and vertices in a graph with edge connectivity n and m n-bonds. *Netw.*, 5(3):253–298, July 1975.

[CR04] L. Sunil Chandran and L. Shankar Ram. On the number of minimum cuts in a graph. *SIAM J. Discret. Math.*, 18(1):177–194, 2004.

[DKL76] Efim A. Dinitz, Alexander V. Karzanov, and Michael V. Lomonosov. On the structure of the system of minimum edge cuts of a graph. *Studies in discrete optimization*, 1976.

[FF09] Tamás Fleiner and András Frank. A quick proof for the cactus representation of mincuts. *EGRES Quick Proof*, 2009-03, 2009.

[GMW20a] Pawel Gawrychowski, Shay Mozes, and Oren Weimann. Minimum cut in $O(m \log^2 n)$ time. In Artur Czumaj, Anuj Dawar, and Emanuela Merelli, editors, *47th International Colloquium on Automata, Languages, and Programming, ICALP 2020, July 8-11, 2020, Saarbrücken, Germany (Virtual Conference)*, volume 168 of *LIPIcs*, pages 57:1–57:15. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020.

[GMW20b] Pawel Gawrychowski, Shay Mozes, and Oren Weimann. A note on a recent algorithm for minimum cut. *CoRR*, abs/2008.02060, 2020.

[GNT20] Mohsen Ghaffari, Krzysztof Nowicki, and Mikkel Thorup. Faster algorithms for edge connectivity via random 2-out contractions. In Shuchi Chawla, editor, *Proceedings of the 2020 ACM-SIAM Symposium on Discrete Algorithms, SODA 2020, Salt Lake City, UT, USA, January 5-8, 2020*, pages 1260–1279. SIAM, 2020.

[Goe06] Michel X. Goemans. Minimum bounded degree spanning trees. In *47th Annual IEEE Symposium on Foundations of Computer Science (FOCS 2006), 21-24 October 2006, Berkeley, California, USA, Proceedings*, pages 273–282. IEEE Computer Society, 2006.

[GPRW20] Andrei Graur, Tristan Pollner, Vidhya Ramaswamy, and S. Matthew Weinberg. New query lower bounds for submodular function minimization. In Thomas Vidick, editor, *11th Innovations in Theoretical Computer Science Conference, ITCS 2020, January 12-14, 2020, Seattle, Washington, USA*, volume 151 of *LIPIcs*, pages 64:1–64:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020.

[GR95] Michel X. Goemans and V. S. Ramakrishnan. Minimizing submodular functions over families of sets. *Comb.*, 15(4):499–513, 1995.

[Har08] Nicholas J. A. Harvey. *Matchings, matroids and submodular functions*. PhD thesis, Massachusetts Institute of Technology, Cambridge, MA, USA, 2008.

[HMT88] András Hajnal, Wolfgang Maass, and György Turán. On the communication complexity of graph properties. In *Proceedings of the 20th Annual ACM Symposium on Theory of Computing, May 2-4, 1988, Chicago, Illinois, USA*, pages 186–191, 1988.

[HRW20] Monika Henzinger, Satish Rao, and Di Wang. Local flow partitioning for faster edge connectivity. *SIAM J. Comput.*, 49(1):1–36, 2020.

[HW96] Monika Rauch Henzinger and David P. Williamson. On the number of small cuts in a graph. *Inf. Process. Lett.*, 59(1):41–44, 1996.

[Jai01] Kamal Jain. A factor 2 approximation algorithm for the generalized Steiner network problem. *Comb.*, 21(1):39–60, 2001.

[Kar93] David R. Karger. Global min-cuts in RNC, and other ramifications of a simple min-cut algorithm. In Vijaya Ramachandran, editor, *Proceedings of the Fourth Annual ACM/SIGACT-SIAM Symposium on Discrete Algorithms, 25-27 January 1993, Austin, Texas, USA*, pages 21–30. ACM/SIAM, 1993.

[Kar00] David R. Karger. Minimum cuts in near-linear time. *J. ACM*, 47(1):46–76, 2000.

[KT19] Ken-ichi Kawarabayashi and Mikkel Thorup. Deterministic edge connectivity in near-linear time. *J. ACM*, 66(1):4:1–4:50, 2019.

[KV18] Bernhard Korte and Jens Vygen. *Combinatorial Optimization: Theory and Algorithms*. Springer, 2018.

[Lov93] László Lovász. *Combinatorial problems and exercises (2. ed.)*. North-Holland, 1993.

[MN20] Sagnik Mukhopadhyay and Danupon Nanongkai. Weighted min-cut: sequential, cut-query, and streaming algorithms. In *Proceedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing, STOC 2020, Chicago, IL, USA, June 22-26, 2020*, pages 496–509, 2020.

[NNI97] Hiroshi Nagamochi, Kazuhiro Nishimura, and Toshihide Ibaraki. Computing all small cuts in an undirected network. *SIAM J. Discret. Math.*, 10(3):469–481, 1997.

[RSW18] Aviad Rubinstein, Tselil Schramm, and S. Matthew Weinberg. Computing exact minimum cuts without knowing the graph. In *9th Innovations in Theoretical Computer Science Conference, ITCS 2018, January 11-14, 2018, Cambridge, MA, USA*, pages 39:1–39:16, 2018.